PRD: MediQue (Clinic Appointment Booking System)

1. Header

Field	Value
Product	MediQue – Clinic Appointment Booking System
ID/Track	PST_AD_057 / Application Developement
Batch / Level	2026 Core
Duration	4 weeks (one-month OJT)
Student/Team	Princy Anindita and Vijaya Lakshmee.
PRD version	v0.2

2. Overview

Project to build MediQue, a clinic appointment booking application. Doctors publish time slots, patients book them, and the system guarantees that a slot can never be booked twice. The project is deliberately small: a front-end application built with HTML, CSS, JavaScript and Bootstrap 5, running on sample data and browser LocalStorage. Booking rules are placed in a separate, testable JavaScript module so that automated tests can prove there is no double booking.

- **Industry:** Healthcare administration / clinic operations
- **Theme:** Appointment scheduling with conflict prevention
- **Product promise:** Pick a doctor, choose a free slot, and book it. No double bookings, with clear screens for patients, front-desk staff and doctors.
- **Primary objective:** Replace phone-and-notebook scheduling with a simple booking flow that is correct on sample data and easy to use.
- **Guiding rule:** Don't overbuild. Finish the core flow and the automated no-double-booking tests first; only then use week 3 for stretch items.

3. Problem statement

Small clinics often schedule appointments by phone, paper diaries or shared spreadsheets. Two staff members can promise the same slot to different patients, patients cannot see which times are free, and doctors do not have a reliable view of their day. MediQue provides doctor slots, patient booking and a guarantee of no double booking, so patients get a confirmed time and the clinic can trust its schedule.

Student fills:

- Primary problem statement and the specific workflow being improved (booking, cancelling or managing a doctor's day).

Small clinics often rely on phone calls, paper registers and manually maintained spreadsheets to manage patient appointments. These methods can make it difficult to track doctor availability, prevent duplicate bookings, manage cancellations and maintain an accurate daily schedule.

This project aims to develop a simple digital appointment management platform that allows patients to find doctors and book available time slots, doctors to manage their availability, and receptionists to coordinate appointments. The system will maintain a consistent appointment record, prevent conflicting bookings and make the scheduling process easier to manage.

The primary workflow being improved is appointment booking and cancellation, supported by doctor availability and receptionist schedule management.

• Three research questions:

(1) **How do clinics** currently book and **avoid clashes**? Clinics currently avoid scheduling clashes by using **integrated software and structured time-block strategies**. The core strategies involve using **centralized scheduling, assigning fixed duration based on visit type, staggered start times,** patients pick directly from live slots via websites or whatsapp, removing human error. **Automated reminders and waitlists confirm or cancel slots early, instantly filling openings from a digital waitlist**.

(2) Which **booking steps cause the most errors** or confusion? **Wrong appointment types**, **duplicate accounts** when booking, **unclear timings**, **last-minute cancellations** that cause inconveniences, race conditions in which **users select the same slot simultaneously** and the slower user receives an error message, all these booking steps cause the most errors/confusion.

(3) What information must each role see to complete its task? **Patients** see provider details, specific check-in vs. doctor times, and cost estimates to book and arrive prepared. **Receptionists and nurses** track live patient statuses, room assignments, billing flags, and incoming notes to coordinate daily workflows. **Doctors** view a clean, sequential timeline of their day paired with **patient medical histories** and **fresh vitals to deliver focused care.**

• Hypothesis: a single booking function that checks and writes in one step, combined with visible slot status, will remove double bookings and reduce booking effort.

• The decision the product supports: which slot a patient should book, and whether a slot can be booked, cancelled or released.

4. Personas

Persona	**Responsibilities**	**Goals**
Patient	Find a suitable doctor, select an available appointment, provide a reason for the visit, manage or cancel appointments.	Book an appointment quickly, know exactly when and where to attend, and avoid booking conflicts.
Receptionist (Front-desk)	Manage walk-in and phone bookings, maintain appointment records, handle cancellations.	Maintain one reliable appointment schedule, find suitable slots quickly, and avoid scheduling conflicts.
Doctor	Publish available time slots, manage daily availability, review appointments, mark appointments completed.	Maintain an accurate daily schedule, avoid overlapping appointments, know which patients are scheduled.

Persona	**Primary need**	**Expected benefit**	**Challenges**	**Permissions**	**Preferred Information**
Patient	Find a doctor, see free slots and book or cancel an appointment	Confirmed time with minimal effort and no clashes	Unclear doctor availability, long waiting times to book by a phone, uncertainty about booking success.	View doctors and available slots, Book appointments, view and cancel their own appointments.	Doctor's name and specialty,available dates and times, booking confirmation and appointment status.

Persona	**Primary need**	**Expected benefit**	**Challenges**	**Permissions**	**Preferred Information**

Receptionist (Front Desk)	Book, cancel and review appointments for walk-in or phone patients	One reliable schedule instead of notebooks	Duplicate bookings, Manually checking multiple schedules, unclear/incomplete patient details.	View all appointments, add or select patients, book and cancel appointments for patients, filter appointments.	Patient details, doctor and appointment time, booking status and date.
Doctor	Publish available slots and view the day's appointments	Accurate daily schedule and control over availability	Unorganised schedules, unavailable or conflicting slots, inaccurate appointment records.	View their own schedule, add and remove eligible slots, mark booked appointments completed.	Patient name. Appointment time, reason for visit, appointment status.

Student task: Create persona cards describing each user's responsibilities, goals, common frustrations, permissions, and preferred information. Define what each role can and cannot do.

5. Key user journeys

Journey 1: Patient books an appointment

1. The patient selects the Patient role (demonstration login).
2. The patient browses doctors, optionally filtered by specialty.
3. The patient selects a doctor and a date and sees slots marked Available or Booked. 4. The patient selects an available slot, enters the reason for the visit and confirms. 5. The system re-checks the slot, saves the appointment and shows a confirmation with doctor, date and time.

Journey 2: Patient cancels an appointment

1. The patient opens My Appointments.
2. The patient selects an upcoming appointment and chooses Cancel.
3. The system asks for confirmation, marks the appointment Cancelled and frees the slot. 4. The slot becomes available to others and the patient sees the updated list.

Journey 3: Doctor manages slots and schedule

1. The doctor selects the Doctor role.
2. The doctor adds available slots (date, start time, duration) or removes unbooked slots. 3. The system rejects slots that overlap the doctor's existing slots.
4. The doctor views the day's schedule and marks completed appointments.

Journey 4: Receptionist books for a patient and hits a conflict

1. The receptionist selects the Front Desk role and chooses an existing patient or enters a new patient's name and phone.
2. The receptionist selects a doctor and an available slot and books it.
3. If the slot has just been taken (for example in another browser tab), the system refuses the booking and shows the latest availability.
4. The receptionist chooses another slot and the booking succeeds.

6. Scope

Must have (MVP – graded)

• Demonstration role selection: Patient, Receptionist, Doctor, with role-specific navigation and screens. • Doctor directory with name, specialty and availability, plus sample doctors.
• Doctor slots: date, start time, end time and status (Available, Booked), with add and remove by the doctor.

• Patient booking: choose doctor, date and slot; provide reason; receive a confirmation. • No double booking: a slot can have at most one active appointment; booking runs through one function that re-checks availability immediately before saving.
• Patient conflict rule: a patient cannot hold two active appointments at overlapping times. • Cancellation with confirmation; a cancelled appointment frees its slot and is kept in history. • Appointment lists: My Appointments (patient), Doctor Schedule (doctor), All Appointments with filters (receptionist).
• Appointment statuses: Booked, Cancelled, Completed.
• Form validation with clear inline errors and success or error messages.
• Sample data: doctors, patients, slots and appointments, including full and empty days. • LocalStorage persistence for the demonstration, with a reset-sample-data option. • Responsive Bootstrap 5 layout for phone and laptop widths.
• Automated tests proving no double booking (see Evaluation).

Good to have (week 3, only if the demo already runs)

• Reschedule an appointment (cancel plus rebook in one safe step).
• Search and filter doctors by specialty or name.
• Week or calendar view of slots.
• Past-slot hiding and 'next available' shortcut.
• Export appointments to CSV.
• Dark theme toggle.
• Optional self-study: port the interface to React + Vite (no MEAN or MongoDB).

Out of scope

• Real backend, database, MEAN stack or MongoDB.
• Real authentication, password management and production-grade security.
• Payments, insurance, billing and prescriptions.
• Electronic medical records or clinical data storage.
• Email, SMS or push reminders (a simulated notice may appear in stretch only). • Video consultations, multi-clinic or multi-branch support.
• Compliance certification (for example full healthcare data regulation).

7. User stories

ID	User story	Acceptance criteria
US-01	As a patient, I want to see doctors and their specialties so that I can choose the right one.	Doctor list shows name, specialty and a link to available slots.
US-02	As a patient, I want to see free and booked slots for a date so that I can pick a time.	Slots show Available or Booked; booked slots cannot be selected.
US-03	As a patient, I want to book a slot so that I have a confirmed appointment.	Booking saves one appointment, marks the slot Booked and shows a confirmation.
US-04	As a patient, I want to be stopped from double booking so that my appointment is not lost.	A second booking of the same slot is rejected with a clear message; an overlapping booking by the same patient is rejected.
US-05	As a patient, I want to cancel an appointment so that the slot is released.	Cancellation needs confirmation, sets status Cancelled and makes the slot Available.

US-06	As a patient, I want to view my appointments so that I know when to attend.	My Appointments lists upcoming and past items with status.
US-07	As a doctor, I want to add and remove slots so that I control my availability.	New slots validate date and time; overlapping slots are rejected; booked slots cannot be removed.
US-08	As a doctor, I want to see my schedule so that I can prepare for the day.	Schedule lists the day's appointments in time order with patient and reason.
US-09	As a doctor, I want to mark an appointment completed so that records stay accurate.	Status changes to Completed only for booked appointments.
US-10	As a receptionist, I want to book and cancel on behalf of a patient so that phone and walk-in patients are handled.	Receptionist can pick or add a patient and book or cancel; the same conflict rules apply.
US-11	As any user, I want clear validation messages so that I can correct mistakes.	Errors appear beside fields, are keyboard accessible and describe the fix.
US-12	As any user, I want the app to work on my phone so that I can use it anywhere.	All main flows work at about 360 px width without horizontal scrolling.

8. Screen inventory

Screen	Key elements	Primary action
Role Select (Demo Login)	Role cards, patient or doctor selector	Enter the app as a role
Doctor Directory	Doctor cards, specialty filter, next available	Choose a doctor
Slot Picker	Date selector, slot grid with Available or Booked states	Select a slot
Booking Form / Confirmation	Patient details, reason, summary, confirm button, success message	Book the appointment
My Appointments (Patient)	Upcoming and past appointments, status badges	View or cancel
Doctor Schedule	Day view of appointments, add-slot form, slot list	Manage slots and complete appointments
Front Desk Appointments	All appointments table, filters by doctor, date, status, patient picker	Book or cancel for a patient

Settings / Reset Reset sample data, current role Reset demo data

9. Component breakdown

Component	Responsibility	Key inputs / events
storage.js	Read and write versioned data in LocalStorage; load sample data	Keys, JSON data, reset
slotService.js	Create, validate, list and remove slots; detect slot overlaps	Doctor ID, date, time, duration
bookingService.js	Book, cancel and complete appointments; enforce no double booking in one synchronous check-and-write	Slot ID, patient ID, reason
validators.js	Validate forms (date not in the past, required fields, phone format, time order)	Form values

roleGuard.js	Decide which actions and screens each role may use	Role, action
Navbar (UI)	Role-specific navigation with collapse on phones	Role, active page
DoctorCard (UI)	Show a doctor and open their slots	Doctor record
SlotGrid (UI)	Render slots with Available or Booked state	Slots, selection
BookingModal (UI)	Collect reason and confirm booking	Selected slot, patient
AppointmentTable (UI)	List appointments with status badges and actions	Appointments, filters
ConfirmModal (UI)	Confirm cancellation	Confirm or cancel event

Toast (UI) Show success and error feedback Message, severity

Student task: Define the data structures, function signatures, error codes and events for each module and write automated tests for the logic modules (storage, slots, booking, validators, role guard).

Data structures: *MediQue will use the following main data entities:*

1. Doctor

1. *doctorId*
2. *name*
3. *specialty*
4. *status*

2. Patient

1. *patientId*
2. *name*
3. *phone*
4. *email (optional)*

3. Slot

1. *slotId*
2. *doctorId*
3. *date*
4. *startTime*
5. *endTime*

4. Appointment

1. *appointmentId*
2. *slotId*
3. *doctorId*
4. *patientId*
5. *reason*
6. *status*

7. *createdAt*
8. *cancelledAt*

5. User (Demo)

1. role
2. doctorId or patientId, depending on the selected roles.

JavaScript Modules:

Module	Responsibility
`main.js`	Initializes MediQue, handles navigation and connects the different application components.
`storage.js`	Handles saving, retrieving and resetting application data using browser LocalStorage.
`slotService.js`	Handles creating, removing and checking the availability of doctor appointment slots.
`bookingService.js`	Handles appointment booking, cancellation and completion.
`validators.js`	Validates booking details, dates, time ranges and user inputs.
`roleGuard.js`	Checks role-based permissions and ensures that users can perform only the actions allowed for their selected role.

Function signatures:

Module	Function Signature	Purpose
`storage.js`	`getData(key)`	Retrieves stored application data.
`storage.js`	`saveData(key, data)`	Saves application data to LocalStorage.
`storage.js`	`resetData()`	Restores the original sample data.
`slotService.js`	`addSlot(slot)`	Creates a new appointment slot.
`slotService.js`	`removeSlot(slotId)`	Removes an eligible appointment slot.
`slotService.js`	`getAvailableSlots(doctorId, date)`	Retrieves available slots for a doctor on a selected date.
`slotService.js`	`hasOverlap(doctorId, date, startTime, endTime)`	Checks for overlapping slots.
`bookingService.js`	`bookAppointment(bookingDetails)`	Validates availability and creates an appointment.

`bookingService.js`	`cancelAppointment(appointmentId)`	Cancels an existing appointment.
`bookingService.js`	`completeAppointment(appointmentId)`	Marks an appointment as completed.
`validators.js`	`validateBooking(bookingDetails)`	Validates appointment booking information.
`validators.js`	`validateSlot(slotDetails)`	Validates appointment slot information.
`roleGuard.js`	`canPerform(role, action)`	Checks whether a role is permitted to perform an action.

Error Handling:

Situation	Expected Behaviour
Slot is already booked	*Prevent the booking and display an error message.*
Patient has an overlapping appointment	*Reject the booking.*
Invalid date or time	*Display a validation message.*
Doctor creates overlapping slots	*Prevent the slot from being created.*
User attempts an unauthorized action	*Display an appropriate error message.*
User attempts to delete a booked slot	*Prevent deletion.*
User attempts to cancel an already cancelled appointment	*Reject the action.*
Data cannot be saved	*Display an error message.*

Automated tests for logic modules:

1. *Booking an available appointment slot.*
2. *Preventing two patients from booking the same slot.*
3. *Preventing a patient from booking overlapping appointments.*
4. *Preventing doctors from creating overlapping slots.*
5. *Cancelling an appointment and making its slot available again.*
6. *Preventing deletion of slots with active bookings.*
7. *Preventing appointments from being booked for past dates.*
8. *Changing a booked appointment's status to Completed.*

10. State and data

Core data entities

Entity	Important fields	Purpose
Doctor	doctorId, name, specialty, status	Stores doctor details
Patient	patientId, name, phone, email (optional)	Stores patient details
Slot	slotId, doctorId, date, startTime, endTime	A bookable time window for one doctor
Appointment	appointmentId, slotId, doctorId, patientId, reason, status, createdAt, cancelledAt	Records a booking and its status
User (Demo)	role, doctorId or patientId	Represents the active demonstration role

The slot's availability is derived: a slot is Booked when it has an active (Booked) appointment, otherwise Available. Do not store a second, independent 'isBooked' flag that can disagree with appointments.

State management

- Application state kept in plain JavaScript objects loaded from LocalStorage and written back on every change.
- UI state: active role, selected doctor, selected date, selected slot, open modal and toast messages. • Persistence: LocalStorage with a version key and a reset-to-sample-data function. • Cross-tab safety: the booking function re-reads storage immediately before writing, and the UI refreshes on the browser 'storage' event.
- Derived values: slot availability, upcoming and past lists, and appointment counts are calculated from records, not stored.

Workflow rules

1. A slot must have a doctor, a valid date, and a start time earlier than its end time. 2. Slots for the same doctor must not overlap.
3. A slot can have at most one active (Booked) appointment.
4. Booking checks the slot's current state from storage, then writes the appointment in the same synchronous function.
5. A patient cannot hold two active appointments that overlap in time, even with different doctors. 6. Slots in the past cannot be booked.
7. Cancelling sets the appointment to Cancelled, keeps the record and frees the slot; cancelled appointments cannot be cancelled again.
8. Only Booked appointments can be Completed; Completed and Cancelled are final states. 9. A slot with an active appointment cannot be deleted.
10. Role checks limit what each role sees and does: patients act only on their own appointments, doctors only on their own slots, and receptionists on all.

Known limitation to document: LocalStorage is not a transactional database and is isolated per browser, so true multi-user, multi-device booking needs a server that enforces a unique constraint on the slot. Front-end role checks are for demonstration, not security.

11. UI states

State	Expected behavior
Loading	Show a brief indicator while sample data loads
Empty	Explain no appointments or slots exist and offer the next action (book, add slot)
Slot Available	Selectable slot with clear label
Slot Booked	Disabled slot labelled Booked (not only a color change)

State	Expected behavior
Success	Toast confirming booking, cancellation, slot added or completion
Booking Conflict	Explain the slot was just taken, refresh slots and suggest another
Overlap Conflict	Explain the patient already has an appointment at that time
Invalid Form	Highlight fields and show actionable messages
Unauthorized Action	Explain the current role cannot perform the action
No Search Results	Explain no doctors or appointments match the filters
Storage Error	Explain data could not be saved or read and offer to reset sample data

12. Responsiveness

Device	Layout	Expected behavior
Phone	Single column, collapsible navbar, slot buttons in a wrapping grid	Browse doctors, book and cancel with thumb-friendly controls
Tablet	Two-column doctor cards and slot grid	Book and review appointments comfortably
Laptop	Navbar, side filters, tables for schedules	Complete every role flow

• Use the Bootstrap 5 grid, utilities and components (navbar, cards, modal, toast, forms, badges, tables). • Wide tables scroll inside a responsive wrapper or become cards on small screens. • Keyboard navigation, visible focus states and touch targets of at least 44 px.
• Test at about 360 px, 768 px and 1280 px.

13. Visual design, accessibility and UX copy

- **Visual style:** Calm, clean, trustworthy; plenty of white space and clear cards.
- **Color:** Neutral base with a healthcare-friendly accent; semantic colors for Available, Booked, Cancelled and Completed that are also distinguished by text or icon.
- **Accessibility:** Semantic HTML, associated labels, accessible modals and toasts, keyboard-operable slot selection, and descriptive status announcements.
- **Contrast:** Target WCAG 2.2 AA.
- **Confirmation:** Always confirm cancellation; show a clear summary before booking. • **UX copy:** Short and specific, for example 'That slot was just booked. Please choose another time.' • **Privacy:** Use only fictional sample patient data; do not collect real health information.

14. Tech stack

- **Stack:** HTML5, CSS3, JavaScript (ES6 modules), Bootstrap 5.
- **Framework:** none beyond Bootstrap; no MEAN stack, no MongoDB.
- **Persistence:** browser LocalStorage with JSON sample data.
- **Testing:** Node.js built-in test runner (node:test) or Jest for the logic modules, kept free of DOM code so they run in the terminal; manual checklist for screens.
- **Local run:** a simple static server (for example the VS Code Live Server extension or python -m http.server), because ES modules do not load from file:// URLs.

• **Version control:** Git and GitHub with clean commits.
• **Deployment:** GitHub Pages or Netlify static hosting, or a reproducible local demo. •
Optional self-study: React + Vite port after the core is complete.

15. Learning goals

• Course covered: Frontend Development Basics.
  • Structure a multi-page or single-page interface with semantic HTML5 and CSS3.

• Build responsive layouts with Bootstrap 5 grid, utilities and components.
• Write modular JavaScript: DOM events, data structures, date and time handling, and LocalStorage. •
Implement and test business rules (conflict prevention) as pure functions.
• Build accessible forms with validation and clear feedback.
• Model a role-based workflow and keep logic separate from UI.
• Use Git and GitHub for version control and collaboration.
• Stretch (optional self-study): React + Vite component thinking.

16. Milestones

Phase	Plan	Expected output
Week 1	Finalize requirements, personas, user journeys and wireframes; Git repository; Bootstrap layout shell; sample data design	Repo, wireframes, responsive shell with role select and navigation
Week 2	Doctor directory, slot model, slot picker, booking and cancellation with LocalStorage persistence	Working book and cancel flow on sample data
Week 3	Role flows (doctor slots and schedule, receptionist view), validation and messages; automated no-double-booking tests. Use any spare time for stretch items only if the demo already runs	Complete role flows and passing core tests
Week 4	Responsive and accessibility pass, bug fixes, more tests, README, screenshots, deployment, demo	MVP release, test results, README, demonstration

Priority: Deliver a working book-and-cancel flow with passing no-double-booking tests before any stretch feature.

17. Evaluation

Catalog metrics:

• No double booking in automated tests.
• Role flows are clear.
• Screens are usable.

Proposed measurable checks (confirm with mentor):

• Automated tests cover at least: two patients booking the same slot, the same booking submitted twice, cancel then rebook, overlapping slots for one doctor, a patient booking overlapping times with two doctors, booking a past slot, and cancelling an already cancelled appointment.
• Role flows: a new user can complete the booking, cancel, add-slot and front-desk booking tasks without help in a short walkthrough with two or three peers, with results noted.
• Usable screens: no horizontal scrolling at about 360 px, visible focus states, and labelled form fields on every screen.

Suggested evaluation rubric

Evaluation area	Weight

Booking logic and no double booking (including tests)	25%
Role flows (patient, receptionist, doctor)	20%
UI quality, responsiveness and accessibility	20%
Code structure, JavaScript quality and data handling	15%
Testing and validation	10%
Documentation and final demonstration	10%
Total	100%

Acceptance criteria:

1. Patients can view doctors and free slots, and book an appointment.
2. The system never creates two active appointments for the same slot.
3. Patients cannot hold two overlapping active appointments.
4. Patients can cancel appointments, and cancelled slots become available again. 5. Doctors can add and remove unbooked slots and view their schedule.
6. Receptionists can book and cancel on behalf of patients.
7. Forms show clear validation and success or error feedback.
8. The interface is usable on phone and laptop widths.
9. Automated tests for the booking rules pass and results are documented.
10. README explains setup, roles, sample data and known limitations.

Expected output: A working ClinicBook web application with doctor slots, patient booking, role-based views, sample data, automated no-double-booking tests, a GitHub repository, README and a deployed or reproducible demo.

18. Risks and assumptions

Risk / assumption	**Potential impact**	**Mitigation**
Scope grows beyond a one-month core OJT	Incomplete core flow	Finish book-and-cancel and tests first; stretch only in week 3 if the demo runs
Availability stored in two places (slot flag and appointment)	Inconsistent double bookings	Derive availability from appointments; single booking function
Two browser tabs book at the same time	Rare clash in LocalStorage	Re-check storage right before writing; refresh on the storage event; document the limitation
LocalStorage cleared or full	Data loss	Reset-sample-data option and clear error messages
Date and time handling errors (time zones, past slots)	Wrong availability	Use local clinic time consistently; test boundaries
Logic mixed into UI code	Hard to test	Keep services DOM-free and test them in Node
Front-end role checks mistaken for security Sample data too limited	Misleading assumption Weak demonstration	State clearly that roles are for demonstration only Include full, empty, past, cancelled and overlapping scenarios
Real patient data entered	Privacy risk	Use fictional data only and show a notice
Accessibility ignored until the end	Poor usability	Check labels, focus and contrast every week

19. Deliverables

- Product requirements document and user journey diagrams.
- Wireframes for all major screens.
- GitHub repository with clean commits and a documented folder structure.
- HTML, CSS, JavaScript and Bootstrap 5 application.
- Role-based views for patient, receptionist and doctor.
- Booking, cancellation and slot management with no double booking.
- Sample dataset with several doctors, patients, slots and appointments.
- LocalStorage persistence with a reset option.
- Automated test suite and test results for the booking rules.
- README with setup instructions, architecture overview, screenshots, role-selection instructions and known limitations.
- Deployed demo or reproducible local demonstration.
- Demo video or live presentation.
    - Optional stretch: reschedule, calendar view, CSV export, dark theme, React + Vite port.

Student task: Document exact tool versions, the sample data structure, the booking and cancellation rules, the test cases and known limitations.

Exact tool versions used:

Tool	Version
HTML	HTML5
CSS	CSS3
JavaScript	ES6
Bootstrap	5.x
Node.js	v26.13.0
Git	git version 2.55.0.windows.5
Code Editor	Visual Studio Code
Browser	Google Chrome Version 156.0.8078.76

Sample Data Structure:

1. **Doctor**

```
{
  doctorId: "D001",
  name: "Dr. Example",
  specialty: "Cardiology",
```

```
  status: "Active"
}
```

2. **Patient**

```
{
  patientId: "P001",
  name: "Sample Patient",
  phone: "9876543210",
  email: "patient@example.com"
}
```

3. **Slot**

```
{
  slotId: "S001",
  doctorId: "D001",
  date: "2026-11-05",
  startTime: "10:00",
  endTime: "10:30"
}
```

4. **Appointment**

```
{
  appointmentId: "A001",
  slotId: "S001",
  doctorId: "D001",
  patientId: "P001",
  reason: "General consultation",
  status: "Booked",
  createdAt: "2026-10-31T10:00:00",
  cancelledAt: null
}
```

Booking and Cancellation rules:

1. *Every appointment slot must belong to a valid doctor.*
2. *The slot must have a valid date and a start time earlier than its end time.*
3. *Slots belonging to the same doctor must not overlap.*
4. *Only one active appointment can be associated with a slot.*
5. *Availability must be checked again immediately before saving an appointment.*
6. *A patient cannot have two overlapping active appointments, even with different doctors.*
7. *Past appointment slots cannot be booked.*
8. *Cancelling an appointment changes its status to Cancelled without deleting its record.*
9. *A cancelled appointment makes its slot available again.*
10. *Only appointments with Booked status can be marked as Completed.*
11. *Slots with active bookings cannot be deleted.*
12. *Users can perform only the actions permitted for their selected demonstration role.*

Test Cases(planned):

Test ID	***Test Scenario***	***Expected Result***
T01	*Book an available appointment slot.*	*Appointment is successfully created.*
T02	*Attempt to book an already occupied slot.*	*Booking is rejected.*
T03	*Two patients attempt to book the same slot.*	*Only one active booking is allowed.*
T04	*A patient attempts to book overlapping appointments with different doctors.*	*Second booking is rejected.*
T05	*A doctor attempts to create overlapping slots.*	*New slot is rejected.*
T06	*Attempt to book a past appointment slot.*	*Booking is rejected.*
T07	*Cancel an active appointment.*	*Status changes to Cancelled and slot becomes available.*
T08	*Attempt to cancel an already cancelled appointment.*	*Further cancellation is rejected.*
T09	*Attempt to delete a slot with an active booking.*	*Deletion is rejected.*
T10	*Mark a booked appointment as completed.*	*Status changes to Completed.*

Known Limitations:

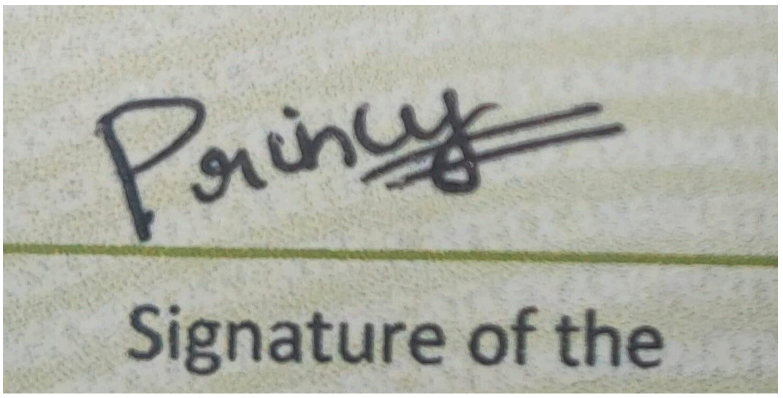
- *Data is stored locally in the browser using LocalStorage.*
- *Data is not shared between different browsers or devices.*
- *The application does not use a backend or central database.*
- *Role selection is for demonstration purposes and does not provide real authentication or security.*
- *LocalStorage does not provide transactional guarantees for simultaneous bookings across multiple devices.*
- *The application does not support online payments, prescriptions, medical records or real-time notifications.*
- *All sample doctors, patients and appointments are fictional.*

20. Sign-off

Role	Name	Date	Approved
Student 1	Princy Anindita	31-10-26	
Student 2	Vijaya Lakshmee	31-10-26	
Mentor / Reviewer	Suhana Jamadar		

Signatures:

- ***Princy Anindita.***



- ***Vijaya Lakshmee***

